

REPORTE TÉCNICO

Clasificación automática del estado de huevos

Entrenamiento, reentrenamiento, persistencia, despliegue en AWS y preparación de pruebas móviles

Estudiante 1	Santiago Perez Florez
ID	U00179515
Estudiante 2	Jose Alejandro Tellez
ID	U00060479
Fecha	24 de septiembre de 2026

Repositorio del proyecto: https://github.com/Joserraldo/damaged_eggs_CNN

1. Resumen ejecutivo

El proyecto tiene como objetivo clasificar visualmente huevos en dos estados: good (sin grieta aparente) y crack (con grieta), integrando un modelo de visión artificial con una API en AWS y una aplicación móvil desarrollada con Expo. Durante el trabajo se ejecutaron dos rondas de entrenamiento, se amplió el conjunto de datos con una segunda fuente de Roboflow, se evaluaron tres arquitecturas y se preparó un nuevo modelo MobileNetV2 para despliegue.

Resultado principal

La segunda ronda utilizó un dataset combinado de 2.249 crops etiquetados. Se entrenaron MLP, CNN y MobileNetV2; el proceso de evaluación seleccionó MobileNetV2 como ganador. El artefacto fue exportado como mejor_mobilenet_v2.keras y empaquetado junto con clases y parámetros de preprocesamiento.

Adicionalmente, se revisó el despliegue existente en una instancia EC2, se verificó que la API FastAPI estaba activa mediante systemd, se creó respaldo del modelo anterior y se cargó el nuevo paquete a la máquina virtual. Para la etapa móvil, se sincronizó el repositorio con los commits más recientes y se preparó la ejecución con Expo Go en iPhone.

2. Objetivos del trabajo

- Entrenar un clasificador binario capaz de diferenciar huevos buenos y huevos agrietados.
- Comparar arquitecturas MLP, CNN y MobileNetV2 bajo un mismo conjunto de evaluación.
- Aumentar la diversidad de datos incorporando un segundo dataset de Roboflow.
- Persistir el modelo ganador en formato Keras junto con su contrato de preprocesamiento.
- Integrar el modelo con la API desplegada en AWS EC2 y dejar lista la prueba desde Expo Go.

3. Arquitectura general de la solución

Componente	Tecnología / función
Fuentes de datos	Roboflow Universe, anotaciones COCO y bounding boxes
Entrenamiento	Google Colab + TensorFlow/Keras
Modelos evaluados	MLP, CNN y MobileNetV2 con transfer learning
Backend	FastAPI + TensorFlow en AWS EC2
Servicio	systemd + Uvicorn en puerto 8000
Aplicación móvil	React Native / Expo / Expo Go
Control de versiones	Git y GitHub

Flujo funcional

Imagen del huevo -> preprocesamiento -> modelo de clasificación -> API /predict -> respuesta good/crack -> visualización en la app móvil.

4. Preparación del dataset y preprocesamiento

El primer entrenamiento utilizó el dataset original del proyecto. Posteriormente se incorporó un segundo dataset de Roboflow denominado Cracked-Eggs. Ambos datasets se descargaron en formato COCO para conservar las bounding boxes y permitir el recorte de cada huevo antes de entrenar.

4.1 Normalización de clases

Etiqueta original	Etiqueta final
good	good
crack	crack
normal-egg	good
cracked-egg	crack
egg (genérica)	ignorar si no aporta estado

4.2 Preprocesamiento aplicado

- Recorte del huevo usando la bounding box COCO con margen aproximado de 4 %.
- Redimensionamiento del crop a 100 x 100 píxeles.

- Conversión a RGB.
- Normalización de intensidades con escala /255.
- Aumentación solo en entrenamiento: flip horizontal, rotación 0.08 y zoom 0.10.

4.3 Dataset combinado V2

Origen	good	crack	Total usable
Dataset anterior	597	642	1.239
Dataset nuevo	497	513	1.010
Total combinado	1.094	1.155	2.249

Split	good	crack	Total
Train	852	913	1.765
Valid	138	113	251
Test	104	129	233

Control de fuga de datos: los crops procedentes de una misma imagen original se mantuvieron dentro del mismo split. Se identificaron 1.911 grupos de imágenes originales para realizar la separación.

5. Entrenamiento y evaluación de modelos

Se entrenaron tres enfoques para comparar complejidad, capacidad de generalización y sensibilidad a la clase crack.

Modelo	Descripción
MLP	Red densa sobre la imagen aplanada; línea base de comparación.
CNN	Convoluciones y pooling entrenados desde cero para aprender patrones locales.
MobileNetV2	Transfer learning con pesos ImageNet y cabeza clasificadora para dos clases.

Callbacks utilizados: EarlyStopping, ReduceLROnPlateau y ModelCheckpoint, tomando val_loss como criterio de seguimiento.

5.1 Primera ronda: línea base

Modelo	Accuracy test	Recall crack	Lectura del resultado
MLP	66,7 %	77,4 %	Desempeño intermedio; no suficientemente robusto.
CNN	61,9 %	96,8 %	Muy sensible a crack, pero con muchos falsos positivos.
MobileNetV2	94,4 %	88,7 %	Mejor equilibrio global entre clases.

En esta primera comparación la CNN obtuvo el mayor recall de crack, pero tendía a etiquetar demasiados huevos buenos como agrietados. MobileNetV2 mostró un comportamiento mucho más balanceado, con mayor exactitud global.

5.2 Segunda ronda: dataset combinado

Tras combinar las dos fuentes de datos y volver a entrenar los tres modelos, la evaluación ejecutada en Colab seleccionó MobileNetV2 como modelo ganador. Las métricas detalladas de esta ronda fueron exportadas en `metricas.json` junto con el modelo. No se fija en este reporte un porcentaje no verificado fuera de la ejecución.

Modelo V2 seleccionado

`mejor_mobilenet_v2.keras` - clasificador binario para good/crack, entrenado con el dataset combinado y el preprocesamiento de crops 100x100 RGB /255.

6. Persistencia y empaquetado del modelo

El modelo ganador se exportó en un paquete preparado para despliegue con los siguientes archivos:

Archivo	Propósito
<code>mejor_mobilenet_v2.keras</code>	Pesos y arquitectura del modelo ganador.
<code>clases.json</code>	Orden de las clases: good y crack.
<code>preprocesamiento.json</code>	Tamaño, RGB, escala /255 y margen de bounding box.
<code>metricas.json</code>	Resultados registrados para los modelos evaluados en la ronda V2.
<code>egg_quality_models_v2.zip</code>	Paquete de transferencia a la VM.

Durante la persistencia se presentaron inconvenientes al escribir en Google Drive desde Colab. Para evitar pérdida del entrenamiento, se generó el ZIP directamente en Colab y se descargó al equipo local.

7. Integración con AWS EC2

La infraestructura AWS ya contaba con una API activa. Se accedió mediante SSH usando la llave `.pem` proporcionada para la instancia y se verificó el servicio `egg-quality`.

Elemento verificado	Estado observado
Servicio systemd	<code>egg-quality.service</code> activo (running)
WorkingDirectory	<code>/home/ubuntu/damaged_eggs_CNN/api</code>
Servidor	Uvicorn / FastAPI en puerto 8000
Modelo previo	<code>/home/ubuntu/egg_quality_models/mejor_mobilenet.keras</code>

Antes de manipular el modelo se creó un respaldo: **`mejor_mobilenet_backup.keras`**

El paquete V2 se transfirió mediante SCP a `/home/ubuntu/egg_quality_models_v2.zip` y se descomprimió en `/home/ubuntu/modelo_v2`. Allí se verificó la presencia de `mejor_mobilenet_v2.keras`, `clases.json`, `metricas.json` y `preprocesamiento.json`.

Estado del despliegue al cierre de este reporte

El nuevo artefacto quedó cargado y disponible en la instancia, con respaldo del modelo anterior. La sustitución definitiva del archivo servido y la validación final del endpoint /health deben confirmarse durante la prueba operativa para asegurar que la API está consumiendo la V2.

8. Preparación de la prueba móvil con Expo Go

Para probar la solución desde iPhone se sincronizó el repositorio local con la rama main. El pull incorporó cambios recientes en mobile/App.js, package.json, package-lock.json y scripts de despliegue/reentrenamiento.

Paso	Acción
1	Entrar a damaged_eggs_CNN/mobile.
2	Instalar dependencias con npm ci.
3	Iniciar Expo con npx expo start -c.
4	Abrir Expo Go en iPhone y escanear el QR.
5	Usar Foto o Galería y enviar la imagen al endpoint /predict.

La aplicación móvil utiliza una URL de API configurable y está diseñada para enviar imágenes JPG/PNG/WEBP a FastAPI. El backend devuelve label, confidence y probabilidades por clase.

9. Hallazgo técnico relevante y trabajo pendiente

Desalineación entrenamiento - inferencia

El modelo V2 descrito en este reporte fue entrenado con crops obtenidos a partir de bounding boxes. Sin embargo, la app móvil puede enviar la fotografía completa del huevo. El repositorio más reciente identifica esta diferencia como una posible causa de errores en producción y añade un script de reentrenamiento con fotos completas.

Por lo tanto, los siguientes pasos recomendados son:

1. Probar la V2 desplegada con imágenes reales capturadas desde el iPhone y registrar errores de clasificación.
2. Reentrenar o ajustar el modelo utilizando imágenes completas, de forma que el dominio de entrenamiento coincida con la entrada de la app.
3. Alternativamente, incorporar una etapa de detección/recorte antes de clasificar, por ejemplo con YOLO, si se desea mantener el clasificador basado en crops.
4. Mantener un test final independiente para evitar ajustar repetidamente el modelo sobre el mismo conjunto de prueba.

10. Incidencias encontradas y soluciones aplicadas

Incidencia	Solución aplicada
Colab perdió variables/rutas temporales	Se redefinieron rutas y se volvieron a descargar datasets cuando fue necesario.

Segundo dataset con permisos distintos de Roboflow	Se utilizó el snippet/API de la cuenta correcta y la ruta ya descargada en /content.
Persistencia en Drive no reflejaba archivos	Se generó el ZIP localmente en Colab y se descargó al PC.
Llave .pem rechazada por permisos de Windows	Se corrigieron permisos y se utilizó una copia accesible para SSH.
git pull bloqueado por package-lock.json no trackeado	Se eliminó el archivo local generado y se repitió el pull.
Expo reportó expo-asset ausente	Se sincronizó el repo y se preparó instalación limpia con npm ci / Expo.

11. Conclusiones

- Se construyó una segunda versión del clasificador utilizando un dataset combinado y balanceado de 2.249 crops.
- MobileNetV2 fue seleccionado como modelo ganador en la segunda ronda de entrenamiento.
- El modelo fue exportado, empaquetado y transferido a la infraestructura AWS existente, manteniendo respaldo del modelo anterior.
- La aplicación móvil y el repositorio fueron sincronizados para preparar pruebas reales mediante Expo Go en iPhone.
- El principal riesgo identificado es la diferencia entre entrenar con crops y predecir sobre fotos completas; esta alineación debe resolverse antes de considerar estable la solución final.

12. Evidencias y artefactos del proyecto

Artefacto / recurso	Uso
clasificador_huevos_colab.ipynb	Entrenamiento y evaluación inicial.
Notebook V2 descargado desde Colab	Entrenamiento con dataset combinado.
egg_quality_models_v2.zip	Modelo V2 y metadatos de despliegue.
api/main.py	API FastAPI de inferencia.
mobile/App.js	Cliente móvil Expo/React Native.
scripts/reentrenar_huevos_colab.py	Flujo reciente para reentrenar con fotos completas.
docs/06-deploy-vm.md	Guía de despliegue en EC2.

13. Comandos representativos del flujo

Etapas	Comando / acción representativa
Sincronización	git pull origin main
Expo	npm ci npx expo start -c
Acceso EC2	ssh -i <llave.pem> ubuntu@<IP-elastica>
Transferencia V2	scp -i <llave.pem> egg_quality_models_v2.zip ubuntu@<IP>:/home/ubuntu/
Verificación servicio	sudo systemctl status egg-quality --no-pager
Verificación API	curl http://localhost:8000/health

Nota final: el modelo desplegado debe conservar exactamente el mismo contrato de entrada usado en entrenamiento; de lo contrario, las métricas de laboratorio no representan el comportamiento real de la app.

14. Evidencia de pruebas en la aplicación móvil

Se realizaron pruebas manuales desde la aplicación móvil conectada a la API desplegada en AWS. Las capturas siguientes documentan respuestas del sistema tanto para huevos con grietas visibles como para huevos sin grietas aparentes. En los ejemplos registrados, la aplicación devolvió predicciones con niveles de confianza entre 52,1 % y 100 %.

Repositorio GitHub utilizado durante las pruebas: https://github.com/Joserraldo/damaged_eggs_CNN

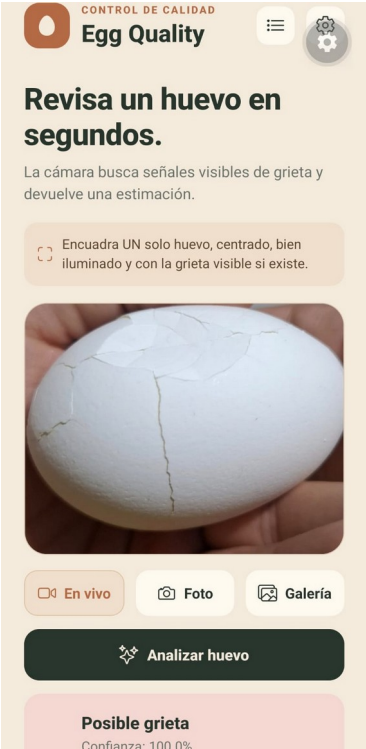
Prueba	Estado visual	Predicción mostrada	Confianza
1	Grieta visible	Posible grieta	99,9 %
2	Grieta visible	Posible grieta	100,0 %
3	Grieta visible	Posible grieta	95,2 %
4	Grieta visible	Posible grieta	82,0 %
5	Sin grieta aparente	Aparenta estar bueno	77,2 %
6	Sin grieta aparente	Aparenta estar bueno	52,1 %
7	Sin grieta aparente	Aparenta estar bueno	79,8 %
8	Sin grieta aparente	Aparenta estar bueno	94,8 %
9	Grieta visible	Posible grieta	100,0 %

Interpretación de la evidencia: las capturas confirman la integración extremo a extremo (iPhone/Expo Go → API en AWS → modelo → respuesta visual en la aplicación). Estas pruebas son demostrativas y no sustituyen una evaluación cuantitativa sobre un conjunto de prueba independiente.

14.1 Casos con grieta visible



Prueba 1 - Grieta detectada. Confianza: 99,9 %.



Prueba 2 - Grieta detectada. Confianza: 100,0 %.



Prueba 3 - Grieta detectada. Confianza: 95,2 %.

14.2 Casos con grieta y transición a casos sin grieta



Prueba 4 - Grieta detectada. Confianza: 82,0 %.

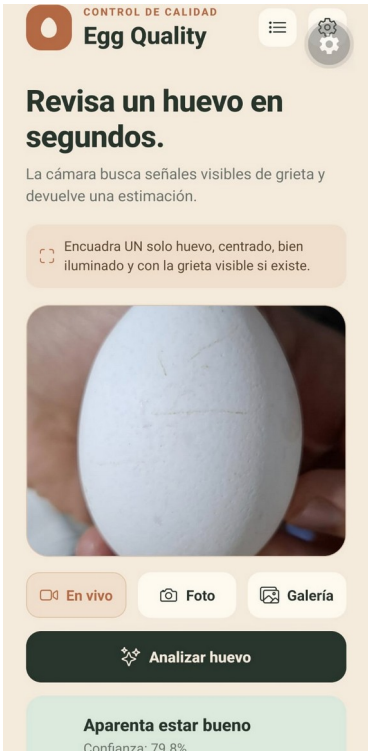


Prueba 5 - Huevo sin grieta aparente. Confianza: 77,2 %.



Prueba 6 - Huevo sin grieta aparente. Confianza: 52,1 %.

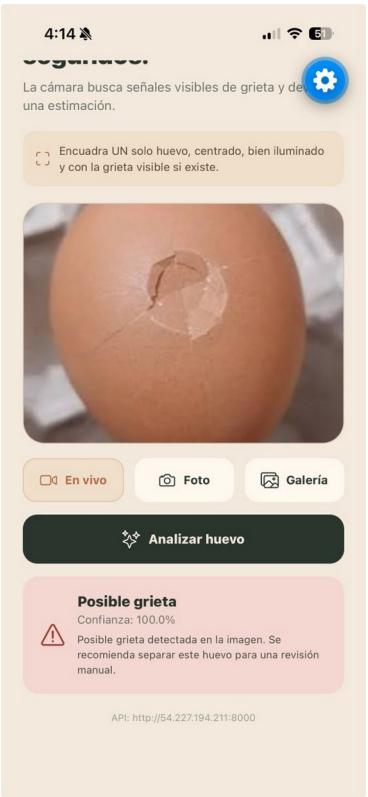
14.3 Casos sin grieta y validación adicional



Prueba 7 - Huevo sin grieta aparente.
Confianza: 79,8 %.



Prueba 8 - Huevo sin grieta aparente.
Confianza: 94,8 %.



Prueba 9 - Grieta detectada. Confianza:
100,0 %.

Fin del reporte